



	Default	Private	Protected	Public
Same Class	Yes	Yes	Yes	Yes
Same Package Subclass	Yes	No	Yes	Yes
Same Package Non-Subclass	Yes	No	Yes	Yes
Different Package Subclass	No	No	Yes	Yes
Different Package Non-Subclass	No	No	No	Yes

what does "non subclass" mean here>



In the context of the table, **"non-subclass"** refers to a class that is not derived (i.e., it does not extend) from the given class. A **subclass** is a class that inherits from another class (its parent class or superclass), while a **non-subclass** does not have this relationship.

For example:

Scenario:

- **Parent class:** The class that defines members (fields/methods) with different access levels.
- **Non-subclass:** A class in the same or a different package that does **not extend the parent class**.

Example in Java:

Parent Class:

```
java

package parentPackage;

public class Parent {
    String defaultField = "Default"; // Default access
    private String privateField = "Private"; // Private access
    protected String protectedField = "Protected"; // Protected access
    public String publicField = "Public"; // Public access
}
```

1. Non-Subclass in the Same Package:

If another class in the same package does not extend the `Parent` class, it is a **non-subclass**.

```
java

package parentPackage;

public class NonSubclassSamePackage {
    public static void main(String[] args) {
        Parent parent = new Parent();
        System.out.println(parent.defaultField); // Accessible
    }
}
```

```

        // System.out.println(parent.privateField); // Not accessible (private)
        System.out.println(parent.protectedField); // Accessible
        System.out.println(parent.publicField);    // Accessible
    }
}

```

2. Non-Subclass in a Different Package:

If the class resides in a different package and does not extend `Parent`, it is also a **non-subclass**.

```

java

package anotherPackage;

import parentPackage.Parent;

public class NonSubclassDifferentPackage {
    public static void main(String[] args) {
        Parent parent = new Parent();
        // System.out.println(parent.defaultField);    // Not accessible (default)
        // System.out.println(parent.privateField);    // Not accessible (private)
        // System.out.println(parent.protectedField);  // Not accessible (protected)
        System.out.println(parent.publicField);       // Accessible
    }
}

```

Summary:

- **Subclass:** Inherits from the parent class (uses the `extends` keyword).
- **Non-subclass:** Does not inherit from the parent class, even if it is in the same package or accesses the class as a standalone entity.

The table shows how access modifiers (`default`, `private`, `protected`, `public`) determine visibility depending on whether the accessing class is a **subclass** or a **non-subclass**, and whether it resides in the same or a different package.